

ПРАКТИЧНЕ ЗАНЯТТЯ №12

Тема: Оброблення виняткових ситуацій. Патерни Facade, Adapter, Dependency Injection, Bridge

Звіт про виконання практичного заняття

Виконав: Вівчар Вадим Вікторович

Група: АЛК-43

Мета роботи

Ознайомитися з принципами оброблення виняткових ситуацій та розглянути структурні й архітектурні патерни Facade, Adapter, Dependency Injection і Bridge. Навчитися пояснювати їх призначення, переваги, структуру та застосування на конкретних програмних прикладах мовою C++.

Теоретичні відомості

Оброблення виняткових ситуацій використовується для реагування програми на помилки, які можуть виникнути під час виконання. До таких ситуацій належать некоректні вхідні дані, відсутність потрібного об'єкта, неправильні параметри, неможливість виконати операцію або порушення логіки роботи програми. У C++ для цього застосовують конструкції try, catch і throw.

Патерн Facade надає простий інтерфейс до складної підсистеми. Він приховує внутрішню послідовність викликів і дозволяє клієнту працювати з одним класом замість набору взаємопов'язаних сервісів.

Патерн Adapter дає змогу використовувати клас із несумісним інтерфейсом у новій системі. Адаптер перетворює виклики клієнта у виклики старого або стороннього об'єкта.

Dependency Injection означає передавання залежностей об'єкту ззовні, а не створення їх усередині класу. Це зменшує зв'язність між класами, полегшує тестування та дозволяє замінювати реалізації без зміни основної логіки.

Патерн Bridge відокремлює абстракцію від реалізації, щоб вони могли змінюватися незалежно. Такий підхід зменшує кількість підкласів і дозволяє комбінувати різні абстракції з різними реалізаціями.

Хід роботи

Для виконання практичного заняття було створено чотири окремі приклади програм мовою C++. Кожен приклад демонструє один із патернів і містить елементи перевірки помилкових ситуацій.

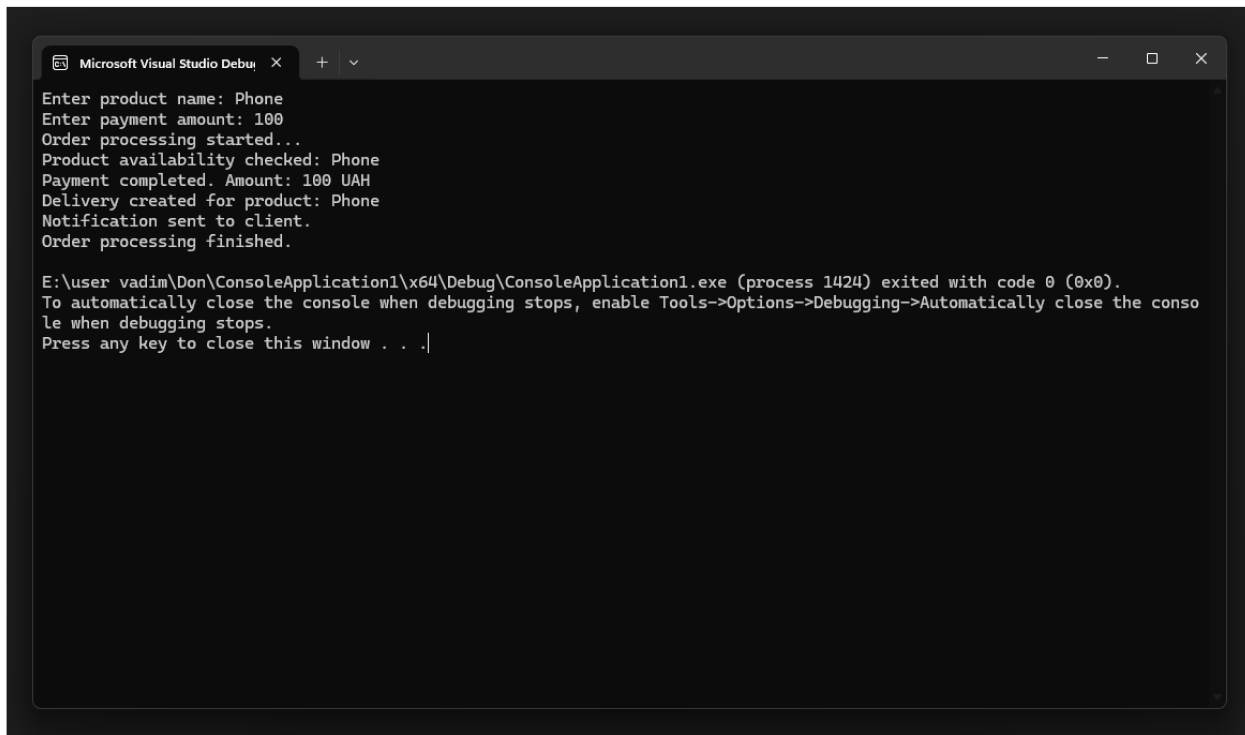
- У першому прикладі реалізовано Facade для оброблення замовлення: перевірка товару, оплата, створення доставки та надсилання повідомлення.
- У другому прикладі реалізовано Adapter, який дозволяє клієнту працювати зі старим принтером через сучасний інтерфейс.
- У третьому прикладі реалізовано Dependency Injection: сервіс створення звіту отримує логер як зовнішню залежність.
- У четвертому прикладі реалізовано Bridge: фігури відокремлені від кольорів, тому їх можна комбінувати незалежно.

Робоча таблиця з відповідями на контрольні запитання

№	Контрольне запитання	Відповідь	Конкретний приклад
1	У чому основна ідея патерну Фасад?	Основна ідея Facade полягає у створенні простого єдиного інтерфейсу для роботи зі складною системою або набором підсистем. Клієнт звертається до фасаду, а фасад сам викликає потрібні компоненти.	Клас OrderFacade має метод processOrder(), який послідовно викликає перевірку товару, оплату, доставку та повідомлення клієнта.
2	Навіщо ховати подробиці системи від клієнта?	Подробиці приховують для спрощення клієнтського коду, зменшення залежності від внутрішніх класів і полегшення супроводу програми. Якщо підсистеми змінюються, клієнт може залишатися без змін.	Клієнт викликає один метод processOrder(), але не знає, які саме сервіси відповідають за оплату, доставку чи повідомлення.
3	Як патерн Фасад декларує розділення системи на підсистеми?	Facade показує, що складну систему можна поділити на окремі підсистеми з різними обов'язками. Фасад не замінює ці підсистеми, а координує їхню роботу через спрощений доступ.	Підсистеми InventoryService, PaymentService, DeliveryService і NotificationService виконують різні завдання, а OrderFacade об'єднує їх у загальний сценарій.
4	Для якого рівня програмної системи варто створювати фасад?	Фасад варто створювати для рівня, де клієнт взаємодіє з кількома складними класами або підсистемами. Найчастіше це рівень сервісів, бізнес-логіки або доступу до зовнішніх ресурсів.	Для модуля замовлень створюється OrderFacade, який приховує роботу з оплатою, складом, доставкою і повідомленнями.
5	Чим обумовлена наявність двох основних варіантів патерну Адаптер?	Два варіанти Adapter пов'язані з різними способами пристосування інтерфейсу: через наслідування або через композицію. Адаптер класу використовує наслідування, адаптер об'єкта містить усередині адаптований об'єкт.	PrinterAdapter містить об'єкт OldPrinter і реалізує інтерфейс ModernPrinter. Метод print() всередині викликає oldPrint().
6	Як можна впроваджувати залежності у патерні Dependency Injection?	Залежності можна впроваджувати через конструктор, метод, властивість або контейнер залежностей. Найпоширеніший спосіб - передавання залежностей через конструктор.	ReportService отримує ILogger через конструктор. Це дозволяє передати ConsoleLogger або іншу реалізацію без зміни ReportService.
7	Як трансформуються залежності при використанні патерну Dependency Injection?	Залежності переносяться зсередини класу назовні. Клас більше не створює конкретні об'єкти самостійно, а отримує їх через інтерфейс або базовий тип.	Замість створення new ConsoleLogger() усередині сервісу клас ReportService працює з інтерфейсом ILogger, а конкретний логер передається ззовні.
8	Що означає відокремлення абстракції від реалізації в патерні Міст? Навіщо це потрібно?	Bridge відокремлює загальну логіку абстракції від конкретного способу її реалізації. Це потрібно, щоб змінювати їх незалежно і не створювати багато класів для кожної комбінації.	Абстракція Shape відокремлена від реалізації Color. Circle, Square і Triangle можуть використовувати RedColor, BlueColor або GreenColor.
9	Чи можливо застосувати декілька ієрархій реалізації відповідно патерну Міст?	Так, можливо. Bridge дозволяє мати незалежні ієрархії абстракцій і реалізацій. Якщо в системі є кілька напрямів змін, їх можна розділити та комбінувати.	Окремо існує ієрархія фігур і окремо ієрархія кольорів. За потреби можна додати ще одну ієрархію, наприклад спосіб виведення на екран або у файл.

Результати виконання програм

Нижче наведено результати запуску програмних прикладів у Visual Studio. Скріншоти підтверджують роботу кожного з розглянутих патернів.

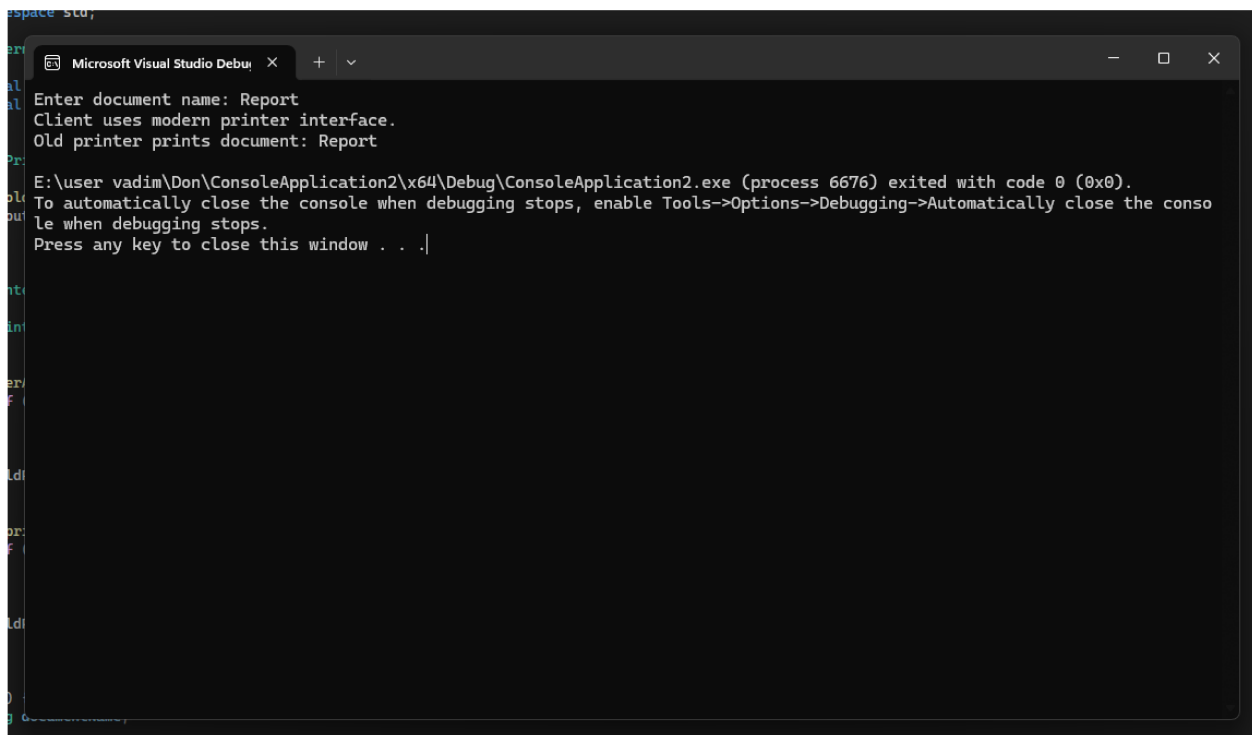


```
Microsoft Visual Studio Debug Console
Enter product name: Phone
Enter payment amount: 100
Order processing started...
Product availability checked: Phone
Payment completed. Amount: 100 UAH
Delivery created for product: Phone
Notification sent to client.
Order processing finished.

E:\user vadim\Don\ConsoleApplication1\x64\Debug\ConsoleApplication1.exe (process 1424) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 1 - Результат виконання програми з патерном Facade

У прикладі Facade клієнт працює з одним об'єктом OrderFacade, який приховує послідовність викликів підсистем замовлення.

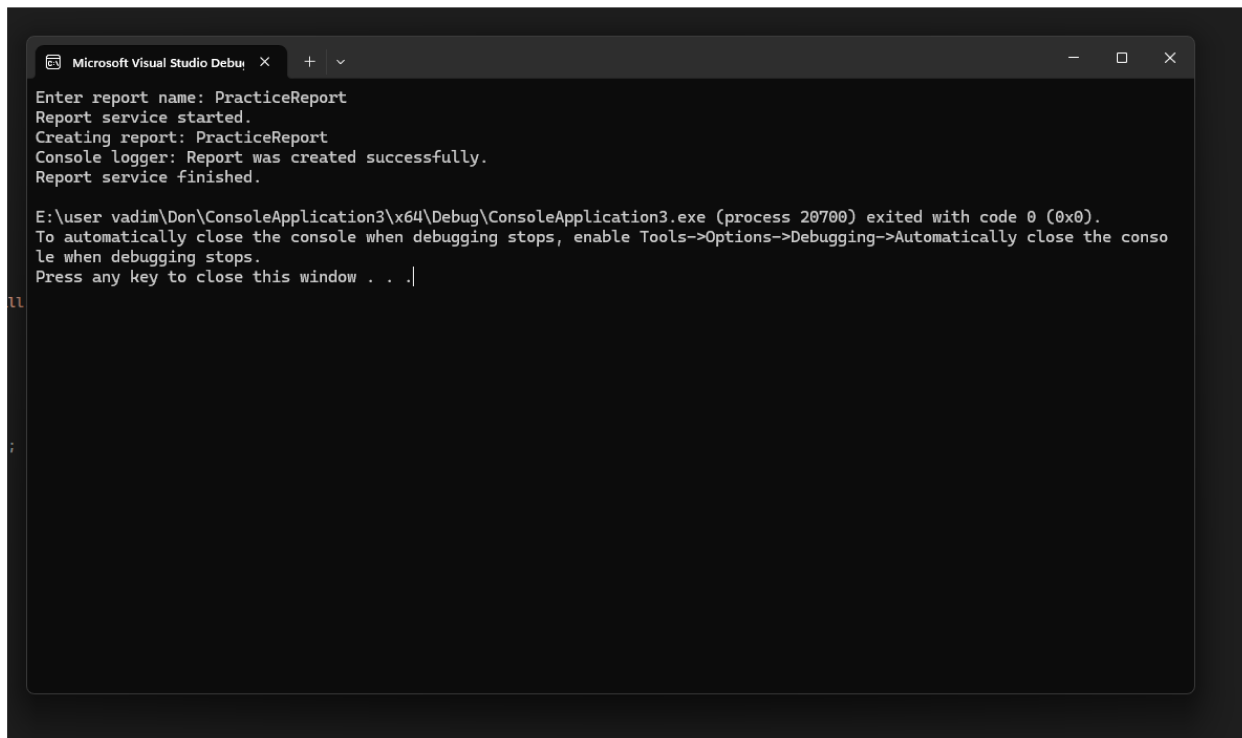


```
Microsoft Visual Studio Debug Console
Enter document name: Report
Client uses modern printer interface.
Old printer prints document: Report

E:\user vadim\Don\ConsoleApplication2\x64\Debug\ConsoleApplication2.exe (process 6676) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 2 - Результат виконання програми з патерном Adapter

У прикладі Adapter клієнт використовує сучасний інтерфейс друку, а адаптер перетворює виклик у метод старого принтера.

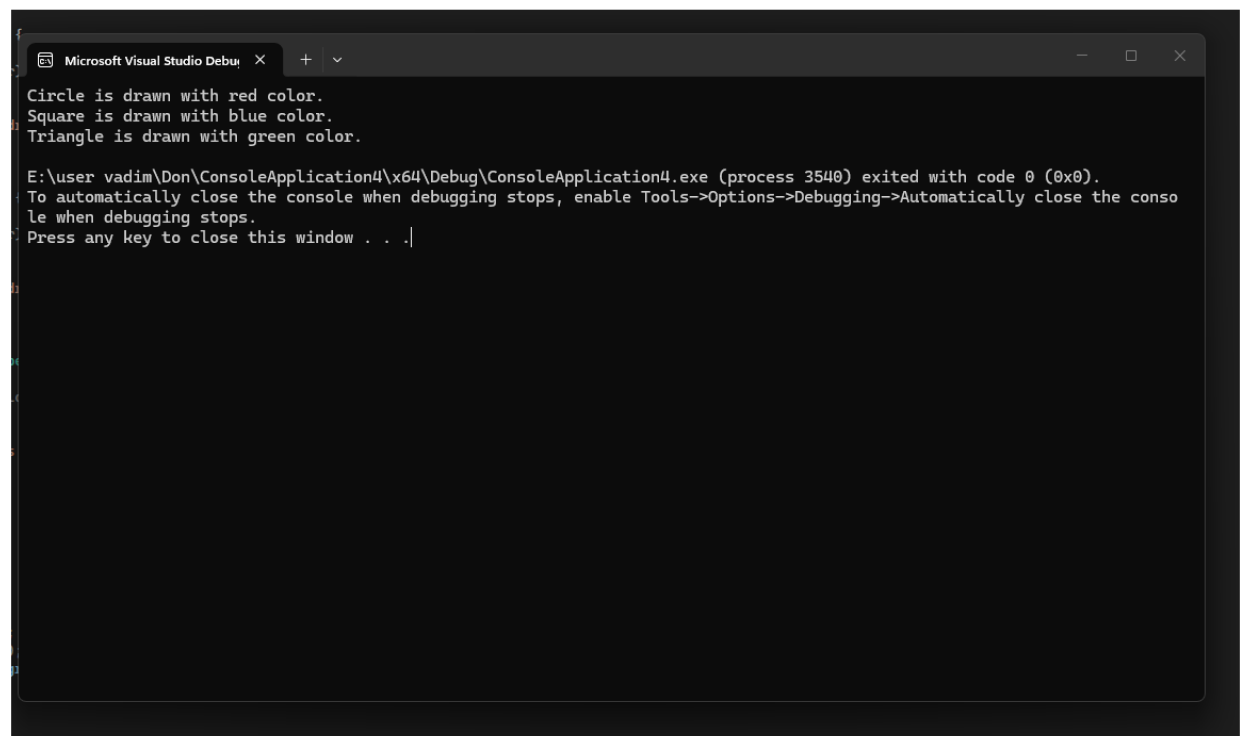


```
Microsoft Visual Studio Debug
Enter report name: PracticeReport
Report service started.
Creating report: PracticeReport
Console logger: Report was created successfully.
Report service finished.

E:\user vadim\Don\ConsoleApplication3\x64\Debug\ConsoleApplication3.exe (process 28700) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 3 - Результат виконання програми з патерном Dependency Injection

У прикладі Dependency Injection сервіс звіту отримує логер ззовні, тому не залежить від конкретного класу логування.



```
Microsoft Visual Studio Debug
Circle is drawn with red color.
Square is drawn with blue color.
Triangle is drawn with green color.

E:\user vadim\Don\ConsoleApplication4\x64\Debug\ConsoleApplication4.exe (process 3540) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рисунок 4 - Результат виконання програми з патерном Bridge

У прикладі Bridge фігури та кольори реалізовані як окремі ієрархії, які можна комбінувати незалежно.

Висновок

У ході виконання практичного заняття було розглянуто патерни Facade, Adapter, Dependency Injection і Bridge, а також їх зв'язок з обробленням виняткових ситуацій.

Було встановлено, що ці патерни допомагають зменшити складність програмної системи, приховати внутрішню реалізацію, адаптувати несумісні інтерфейси та зменшити залежність між класами.

Практичні приклади показали, що Facade спрощує доступ до складної підсистеми, Adapter дозволяє використовувати старий клас у новій системі, Dependency Injection переносить створення залежностей назовні, а Bridge відокремлює абстракцію від реалізації. Такі підходи роблять програмний код гнучкішим, зрозумілішим і зручнішим для подальшого розширення.